

作业 1 Unix 进程、文件和用户

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博 学号 2351520

习题 1: Unix V6++ 进程控制块包括一个 Process 结构和一个 User 结构。抄写 Process、User 结构定义，注释每个字段。酌情加入自己的理解。

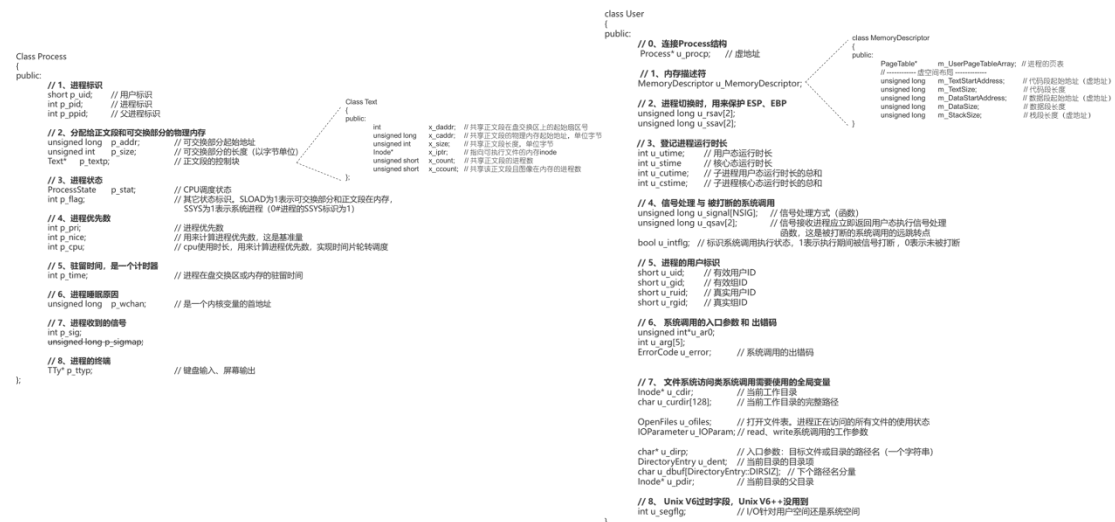


图 1、Unix V6++ 的进程控制块

class Process {

public:

// 1. 进程标识

short p_uid; // 用户标识 (User ID)。标识该进程属于哪个用户。操作系统根据用户 ID 来判断进程对文件和其他资源的访问权限。

int p_pid; // 进程标识 (Process ID)。唯一标识一个进程的编号。

int p_ppid; // 父进程标识 (Parent Process ID)。创建该进程的父进程的 PID。这构成了系统的进程树状结构。

// 2. 分配给正文段和可交换部分的物理内存

unsigned long p_addr; // 可交换部分起始地址。包括 PPDA，数据段和用户栈。

unsigned int p_size; // 可交换部分的长度（以字节为单位）

Text *p_textp; // 指向正文段控制块的指针，指向一个 Text 结构体，该结构体描述了进程的代码段（正文段）。通过这种方式，多个进程可以共享同一个代码段，以节省内存。

// 3. 进程状态

ProcessState p_stat; // CPU 调度状态

int p_flag; // 其他状态标识。SLOAD 值为 1 时，表示进程的核心部分（可交换部分和正文段）当前位于主内存中。SSYS 值为 1 时，表示这是一个系统进程，例如调度进程或交换进程，它们不能被换出。

```

// 4. 进程优先数
int p_pri;          // 进程优先级，数值越小，优先级越高。调度程序会选择优先级最高的就绪进程来运行。
int p_nice;         // 用于计算进程优先级的基准量
int p_cpu;          // CPU 使用时长的记录

// 5. 驻留时间，是一个计时器
int p_time;         // 进程在盘交换区或内存的驻留时间

// 6. 进程睡眠原因
unsigned long p_wchan; // 如果进程处于睡眠状态，p_wchan 指向它正在等待的内核事件的地址。

// 7. 进程收到的信号
int p_sig;          // 进程收到的信号

// 8. 进程的终端
TTY *p_ttyp;        // 键盘输入、屏幕输出的控制终端指针
};

Class Text
{
public:
    Int x_daddr;
    //共享正文段(text 结构)在盘交换区上的起始扇区号
    Unsigned long x_caddr;
    //共享正文段的物理内存起始地址，单位为字节
    Unsigned int x_size;
    //共享正文段的长度
    Inode* x_iptr;
    //指向可执行文件(FCB)
    Unsigned short x_count; //共享正文段的进程数(同一个应用(代码段)有多个进程)
    Unsigned short x_ccount; //共享该正文段且图像在内存的进程数
}

class User {
public:
    // 0. 连接 Process 结构
    Process *u_procp;    // 指向 Process 结构的指针

    // 1. 内存描述符
    MemoryDescriptor u_MemoryDescriptor; // 这是一个结构体，详细描述了进程的虚拟地址空间布局，包括代码段、数据段和栈的起始地址与大小

```

```

// 2. 进程切换时，用来保护 ESP、EBP
unsigned long u_rsav[2];
unsigned long u_ssav[2];

// 3. 登记进程运行时长
int u_utime;        // 用户态运行时长
int u_stime;        // 核心态运行时长
int u_cutime;       // 子进程用户态运行时长的总和
int u_cstime;       // 子进程核心态运行时长的总和

// 4. 信号处理与被打断的系统调用
unsigned long u_signal[NSIG]; // 信号处理方式
unsigned long u_qsav[2];      // 信号接收进程应立即返回用户态执行信号处理函数，
// 这是被打断的系统调用的远跳转点
bool u_intflg;               // 标识系统调用执行状态。1 表示系统调用被打断，0 表示未被打断。

// 5. 进程的用户标识
short u_uid;                // 有效用户 ID
short u_gid;                // 有效组 ID。用于权限检查。通常，它与真实 ID 相同，但在执行
// 设置了 set-user-ID 权限位的程序时，有效 ID 会临时变为程序文件的所有者 ID。
short u_ruid;               // 真实用户 ID
short u_rgid;               // 真实组 ID

// 6. 系统调用的入口参数和出错码
unsigned int *u_ar0;
int u_arg[5];
ErrorCode u_error; // 系统调用的出错码

// 7. 文件系统访问类系统调用需要使用的全局变量
Inode *u_cdir;        // 当前工作目录的内存 i-node
char u_curdir[128];   // 当前工作目录的完整路径
OpenFiles u_ofiles;   // 打开文件表，进程正在使用的所有文件的状态
IOParameter u_IOParam; // read, write 系统调用的工作参数

char *u_dirp;         // 入口参数，目标文件或目录的路径名
DirectoryEntry u_dent; // 当前目录的目录项
char u_dbuf[DirectoryEntry::DIRSIZ]; // 下个路径名分量
Inode *u_pdir;        // 当前目录的父目录 i-node

// 8. Unix V6 过时字段
int u_segflg;
};

```

一、Unix V6++ 进程图像 & 地址

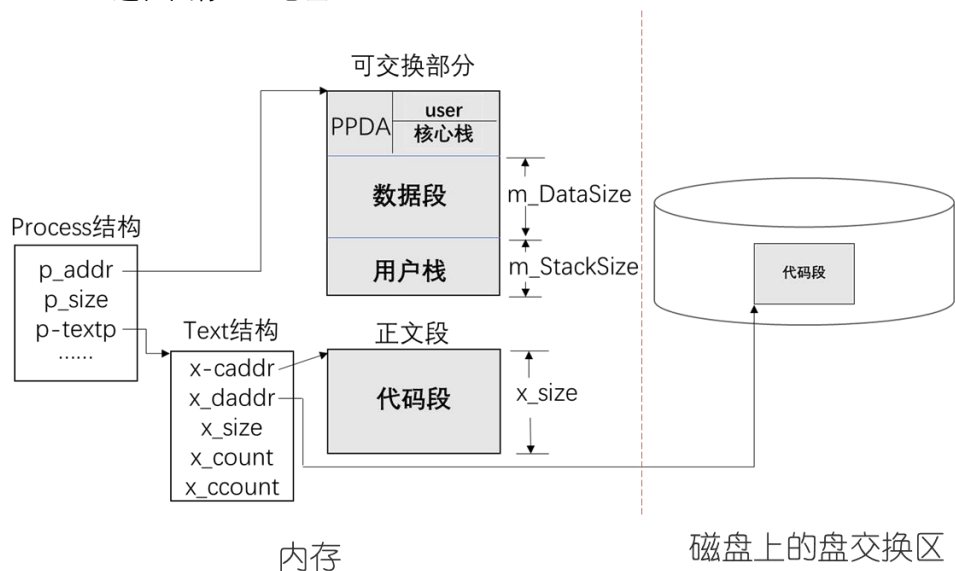


图 2、物理空间中的 Unix V6++ 进程图像

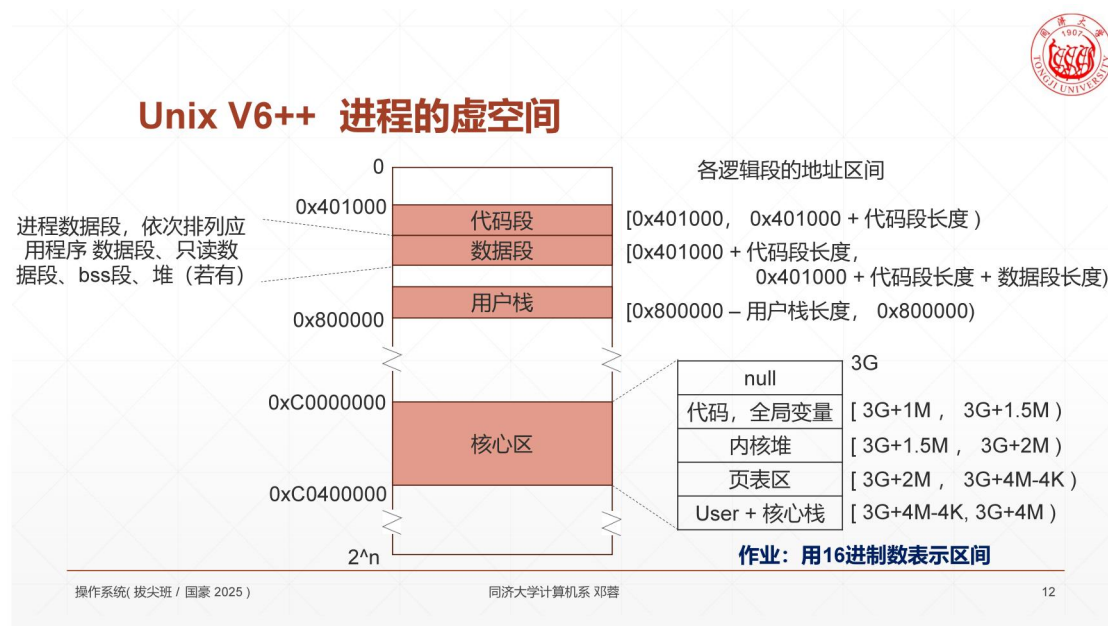


图 3、虚空间中的 Unix V6++ 进程图像

所有逻辑段 4096 字节对齐 (首地址是 4096 字节的整数倍, 长度是 4096 字节的整数倍)

习题 2: Unix V6++ 系统。进程 PB, 3 页代码, 1 页数据, 1 页 bss, 2 页堆栈。问,

- (1) 虚空间中, 代码段首地址=(**0x401000**), 数据段首地址=(**0x404000**),
bss 段首地址=(**0x405000**), 用户栈首地址=(**0x7FE000**)。

(2) 物理空间中,

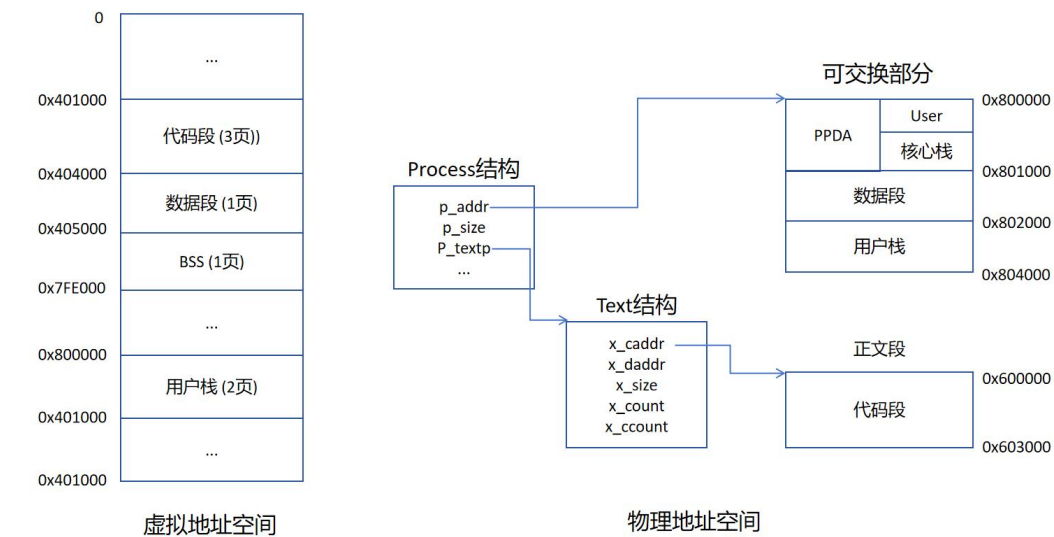
代码段长度 x_size =(**0x3000**), 可交换部分长度 p_size =(**0x5000**)

已知 $x_caddr = 0x600000$, $p_addr = 0x800000$, 求物理空间中

代码段首地址=(**0x600000**), 数据段首地址=(**0x801000**),

bss 段首地址=(**0x802000**), 用户栈首地址=(**0x803000**)。

(3) 仿照图 2、图 3, 画进程 PB 图像, 求进程 PB 的页表。



映射用户空间的页表

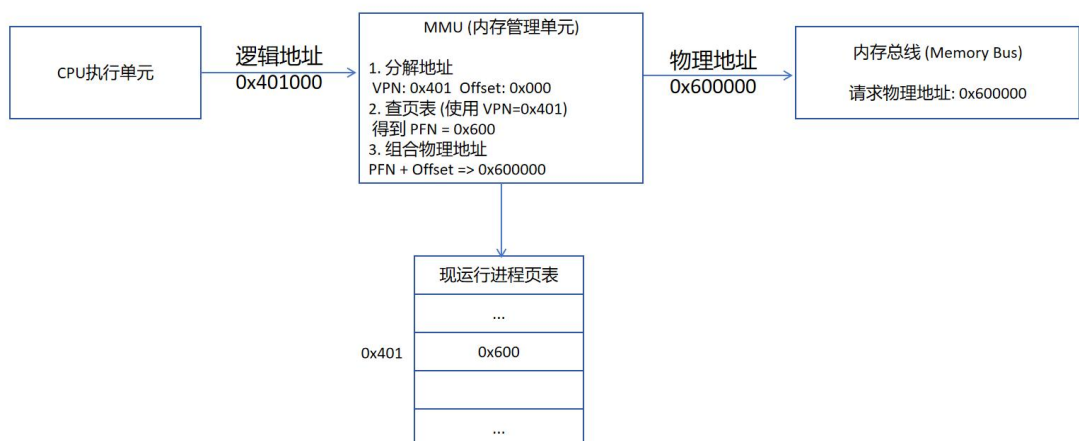
0	null
1~0x400	null
0x401	0x600
0x402	0x601
0x403	0x602
0x404	0x801
0x405	0x802
0x406~0x7fd	null
0x7fe	0x803

映射内核空间的页表

0xC0000	0
0xC0001	1
...	...
0xC03ff	0x800

(4) 求 指令、数据地址。

4.1 PB 进程取代码段第 1 条指令。CPU 执行单元给出虚地址=(0x401000), MMU 地址映射后生成的物理地址=(0x600000)。图示 MMU 地址映射过程。



习题 3: Unix V6++系统。已知 matrix 进程，代码段 3 页。

x_caddr=0x500000, p_addr=0x600000。求

```
# include <stdlib.h>
# include <stdio.h>

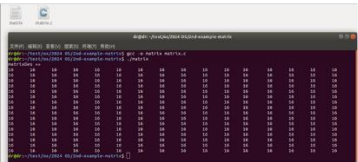
# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];

void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

int main()
{
    int i , j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i , j);

    printf("matrixDes == \n");
    for (i=0;i<M;i++) {
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
            printf("%d\t", matrixDes[i][j]);
        }
        printf("\n");
    }
}
```



- (1) 数据段长度=(0x1000), 用户栈长度=(0x1000), 可交换部分长度=(0x5000)。
- (2) 选择、填空：
- main 函数存放在(a)。
 - version 变量是(e), 存放在(b), 虚地址=(0x404000), 物理地址=(0x601000)。
 - 矩阵 matrixOriginal 是(f), 存放在(c), 虚地址=(0x406000), 物理地址=(0x603000)。
 - 矩阵 matrixDes 是(f), 存放在(c), 虚地址=(0x406400), 物理地址=(0x603400)。
 - 矩阵元素 matrixDes[2][2] 虚地址=(0x406488)。 物理地址=(0x603488)。
 - 访问寄存器 EAX。CPU 执行单元给出虚地址=(CPU 寄存器 (如 EAX) 位于 CPU 内部，而不是在内存中。因此，它们没有内存地址 (无论是虚拟还是物理地址)), MMU 地址映射后生成的物理地址=()。

红色是单选题。选填 (a) 代码段 (b) 数据段 (c) bss 段 (d) 堆栈段 (e) 带初值的全局变量 (f) 不带初值的全局变量 (g) 局部变量。
黑色是填空题。

二、进程 uid 和 文件 uid

习题 4: 某 Linux 系统，10 台终端。花名册文件内容如下：

user name	password	uid	gid	PATH	initial working directory	initial shell
root	X	0	0	*****	/root	/bin/sh
.....						
John	X	12	1	*****	/home/John	/bin/sh
Mary	X	13	1	*****	/home/Mary	/bin/sh
.....						
Tony	X	58	1	*****	/home/Tony	/bin/x_windows

T 时刻, Tony 用户使用 7#终端, Mary 用户使用 5#终端, 如图:

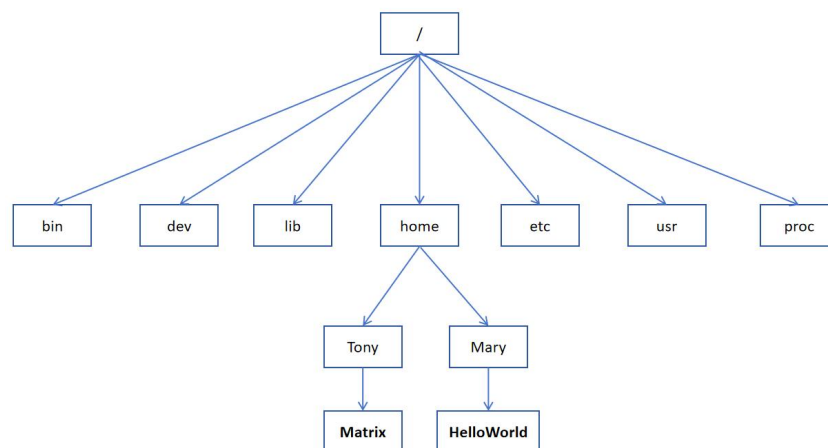
Login: Tony
Password: *****
.....没碰过cd.....
\$ gcc -o Matrix Matrix.c

用 7# 终端

Login: Mary
Password: *****
.....没碰过cd.....
\$ gcc -o HelloWorld HelloWorld.c

用 5#终端

- (1) 在目录树上标出可执行文件 Matrix 和 HelloWorld 的位置,
确定可执行文件 Matrix 的绝对路径名和 uid,
确定可执行文件 HelloWorld 的绝对路径名和 uid,



Matrix 的绝对路径名: /home/Tony/Matrix uid=58

HelloWorld 的绝对路径名: /home/Mary/HelloWorld uid=13

- (2) 这 2 个 gcc 进程, p_uid 和 u_cdir ? p_ttyp?
Matrix: p_uid = 58; u_cdir --> home/Tony; p_ttyp-->tty7
HelloWorld: p_uid = 13; u_cdir --> home/Mary; p_ttyp-->tty5
- (3) 一周后, Tony 重新编译 Matrix.c 程序。为之服务的进程, p_uid 和 u_cdir ?
p_ttyp ?
p_uid 和 u_cdir 不变, p_uid = 58, u_cdir --> home/Tony;
p_ttyp 不知道, 因为不知道 Tony 使用的终端。

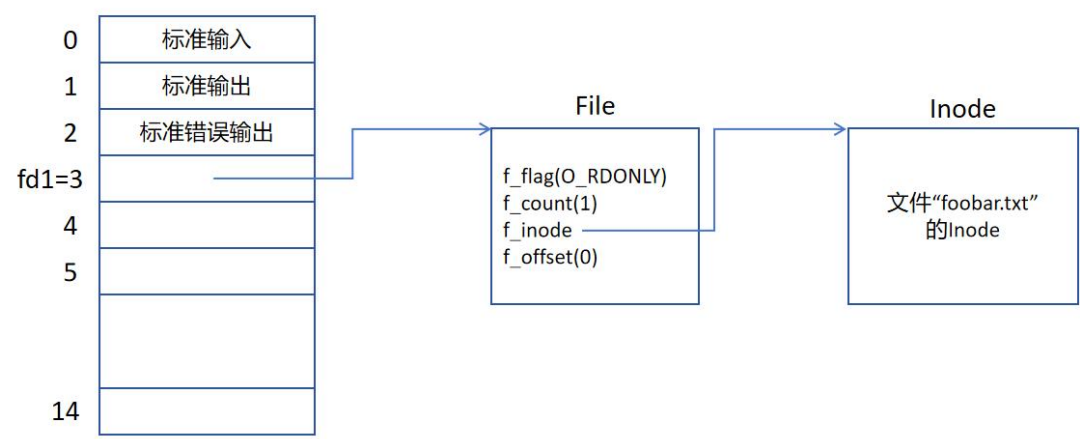
三、进程打开文件结构 和 读写操作

习题 4: 画语句 1、2、3、4 执行结束后, example2 进程的打开文件结构

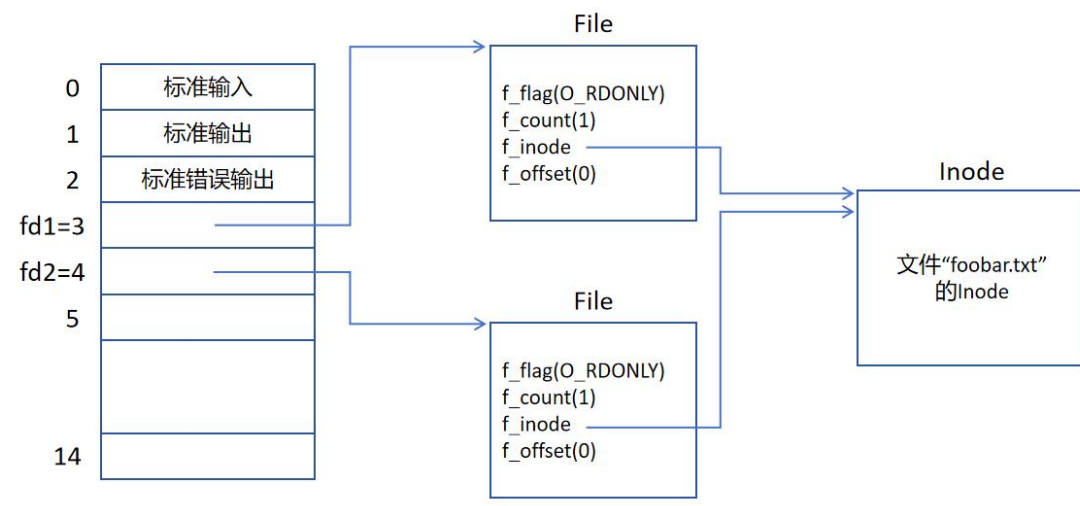
```
int main() // example2.c
{
    int fd1, fd2;
    char c;

    1. fd1 = Open("foobar.txt", O_RDONLY, 0);
    2. fd2 = Open("foobar.txt", O_RDONLY, 0);
    3. Read(fd1, &c, 1);
    4. Read(fd2, &c, 1);
    printf("c = %c\n", c);
    exit(0);
}
```

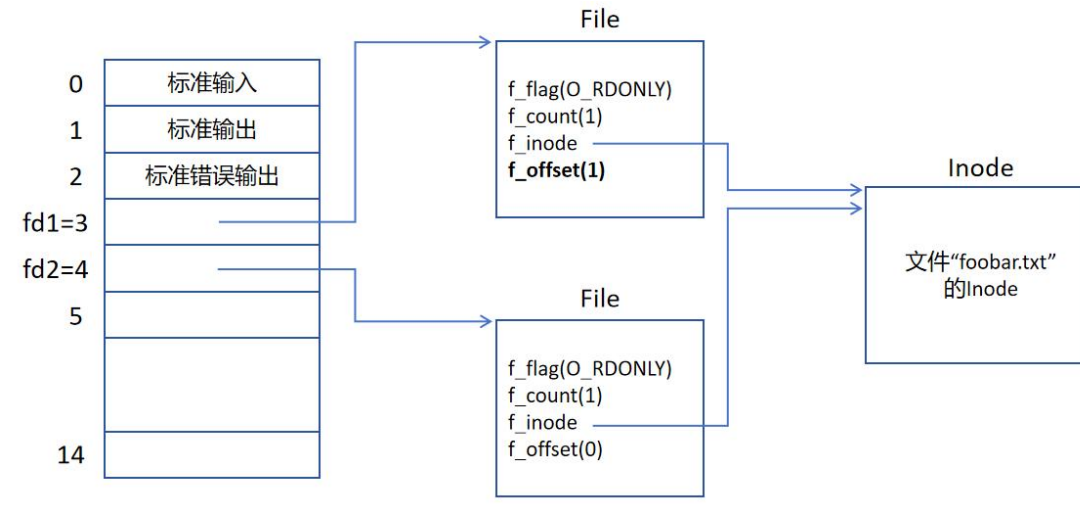
语句 1 执行完：



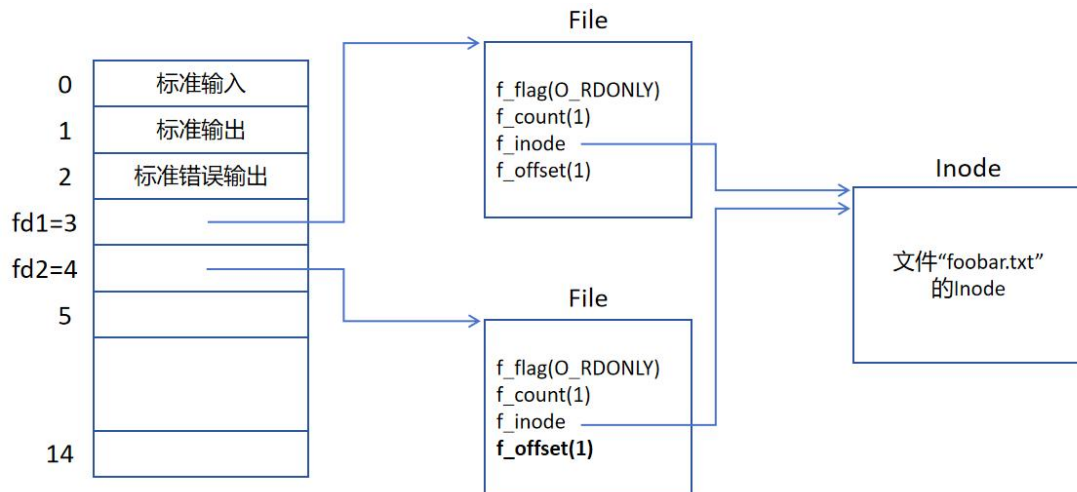
语句 2 执行完：



语句 3 执行完：



语句 4 执行完：



习题 5：磁盘文件 file1，长度是 700 个字节。0~99 字节是 100 个字符 A，100~199 字节是 100 个字符 B，200~299 字节是 100 个字符 C……。

```
int fd = open(file1, RDWR); //读写打开
char data[300];
lseek(fd, 500, 0);
int count = read (fd, data, 300);
count = write(fd, data, 300);
```

问：这段程序执行完毕后，文件 file1 的长度是(**1000 字节**)，文件的内容？

文件内容：

字节 0 ~ 499：保持不变 (依然是 100 个'A', 100 个'B', 100 个'C', 100 个'D', 100 个'E')。

字节 500 ~ 699：保持不变 (依然是 100 个'F', 100 个'G')。

字节 700 ~ 799：新写入的 100 个字符 'F'。

字节 800 ~ 899：新写入的 100 个字符 'G'。

字节 900 ~ 999：新写入的 100 个字节的未定义内容。

四、fork 系统调用

例题 1：下面的这个程序会输出几个整数？请写出程序的输出，并请在代码中标出父进程执行的所有语句和子进程执行的所有语句。

```
L1: #include <stdio.h>
L2: void main (void )
L3: {
L4:     int i = 10, x;           //父
L5:     if ( x = fork() )       //父, 子
L6:     {
L7:         i += 10;            // 父
L8:         printf ( "%d\t", i ); // 父
L9:     }
L10:    else
L11:        printf ( "%d\t", i ); //子
L12:    printf ( "%d\t", i );     //父, 子
```

L13: }

父进程输出 2 个 20，子进程输出 2 个 10，总共会输出 4 个整数。

程序的输出有多种可能，因为父子进程是并发执行的，操作系统调度器决定了哪个进程在哪个时刻获得 CPU 时间。它们的 printf 调用可以任意交错。

可能是： 10 20 10 20

可能是： 10 20 20 10

可能是： 20 10 20 10

可能是： 20 10 10 20

可能是： 20 20 10 10

可能是： 10 10 20 20

题目中提到的极端情况可能是因为当父子进程共享同一个标准输出时，它们各自往缓冲区里写东西，有可能缓冲区刚好满了如何刷新，导致父进程的输出丢失，只输出 10 10

例题 2：阅读下面这段代码。父进程 P0 从 main()入口开始跑，先后创建 2 个子进程 P1、P2。请在代码中标注子进程 P1、P2 执行的代码片段。

```
int main( )
{
    .....
    int pid = fork ( ) ;           // P0
    if (pid == 0)                  // P0、P1
        代码片段 1;                // P1
    else {
        pid = fork ( ) ;          // P0
        if (pid == 0)              // P0、P2
            代码片段 2;            // P2
        else
            代码片段 3;            // P0
    }
}
```